Serving Map Tiles from a Raspberry Pi

A Portable Tile Server Workshop

FOSS4G Hiroshima 2026 / 2026-08-30 14:00–17:00 / Room 610

Communication: Slack

- All announcements, questions and links go through Slack today
- Invitation QR is on screen (also sent by email)
- Anything I look up during the session will be posted there in real time
- Questions posted on Slack will be picked up in the Q&A slot

Translation and materials

- I will speak in Japanese; an English translation (Gemini) is shown on screen
- The translation log will be downloadable afterwards
- **Please download the PDF of these slides now**
- Reason: in Part 3 this room goes offline — on purpose

About me

(shown from a separate deck)

- Freelance in Japan: remote sensing planning, sensing device and app development, VR content, Fab space construction support
- Raspberry Pi user since my student days. Heavy use in field observation and monitoring
- The 7,500 JPY fee goes entirely to conference operations, not to me

Why this workshop exists

- FOSS4G comes to Hiroshima at just the right moment, thanks to the LOC team's efforts
- Predecessors exist: the **UNVT Portable** project by Hirasawa and others
  — the idea of carrying tiles on Raspberry Pi has already been put into practice
- I have been using Raspberry Pi in the field since my student days
  - Honeybee observation system
  - River water gate field monitoring
- Today you will build and hold the whole pipeline in your hands

Motivation: where the network ends

The places that need maps most are the places without connectivity.

- Mountain huts — hikers need surrounding terrain and route information
- Disaster and conflict zones — connectivity cannot be relied on
- Polar and remote regions — satellite connectivity is expensive and limited

So: a tile server you can carry in as a box.

PWA and offline caching comparison

	PWA / app-side caching	Portable tile server
Setup	Required per device	Not required (just connect to Wi-Fi)
Target devices	Prepared devices only	All devices on the network
Data updates	Re-distribute per device	One file swap on the server
Storage limits	Device storage / OS constraints	Depends on SD card (tens of GB)
Best for	Individual solo treks	Centralized operations with groups

No app installs, no per-device caching — everyone connected gets the map.

Today's goals

1. Serve PMTiles with Martin on a Raspberry Pi and view it in a browser
2. Inspect and control the server from the CLI (systemd / curl / journalctl)
3. Update served data by swapping a single file
4. Turn the Pi into a Wi-Fi AP and serve maps fully offline

> Fair warning: at the end, we really do pull the plug.

The big picture

```
[Part 1] Theory: why, what, and how
    |
[Part 2] SSH from your PC to the Pi
    PC --(Opal router)--> Raspberry Pi (Martin + PMTiles)
    |
[Part 3] The Pi itself becomes a Wi-Fi AP
    Smartphone --(foss4g-pi-NN)--> Raspberry Pi   ※offline
```

- Through Part 2: the Pi is "a server on the network"
- From Part 3: the Pi is "the network itself + server"

A short history of Raspberry Pi

- Launched 2012 by the Raspberry Pi Foundation as an educational computer at 25–35 USD
- Evolved far beyond education: now a de-facto standard in industry, hobby, and research
- Lineage: Pi 1 → 2 → 3 → 4 → 5, plus compact variants (Zero / A+) and industrial modules (Compute Module)
- Cumulative sales in the tens of millions. Knowledge, examples, and troubleshooting cases have accumulated worldwide
  — the value of "when stuck, search and find someone else in the same hole" is enormous

The Raspberry Pi family

Line	Example	Traits	Good for
B series	Pi 4B / 5	More RAM, wired LAN, many USB ports	Permanent servers, desktops
A series	**3A+**	Compact, low power, minimal ports	Embedded systems, today's star
Zero series	Zero 2 W	Smallest, cheapest	Sensor nodes, wearables
CM series	CM4 / CM5	Board-mountable, industrial	Product integration

The same OS and toolchain run across all variants, so you can develop on B series and deploy on A or Zero.

What can it do?

Examples I have run in the field:

- **Honeybee observation system** — attach camera and sensors to hives to log behavior
- **River water gate monitoring** — long-term surveillance of gates in low-connectivity environments

For performance perspective:

- Pi 4 with ffmpeg transcoding + HLS streaming: about 3 watts
- 3 W is realistic for battery or solar operation

Tile serving is far lighter than video processing, so today's task has plenty of headroom.

Today's star: Raspberry Pi 3 Model A+

Item	Spec
SoC / RAM	BCM2837B0 (4 core, 1.4GHz) / **512MB**
Wireless	CYW43455: 2.4GHz + **5GHz** Wi-Fi, BT
Ports	USB-A x1, HDMI, microSD
Wired LAN	**None**
Power	microUSB 5V/2.5A

Constraints are the teaching material:

- RAM 512MB → heavy stacks won't fit → **reason to choose a light server**
- No wired LAN → forces thoughtful wireless design
- 5GHz support → pays off in Part 3 when we create the AP

SD cards and power

Raspberry Pi operational failures cluster around these two, by experience.

SD Card

- All OS and data live on the SD card. Quality differences are stark
- Selection guide: trusted brand / **A1 (Application Performance Class) or better**
- Today's setup: 16–32GB, A1. Reduced writes via disabled swap and tmpfs logs

Power

- Voltage drop is the king of mysterious failures. Cable quality matters too
- Today's setup: 5V/2.5A single-port adapter + microUSB cable

Hardware is circulating — please have a look.

How the OS gets there

- The Raspberry Pi boots a disk image written to a microSD card
- **Raspberry Pi Imager** (official GUI tool) makes it simple:
  - OS selection through SD writing all in GUI
  - Pre-configure SSH, Wi-Fi, hostname, and user at write time
    (= "headless" setup: no monitor, no keyboard, go straight to wireless)
- CLI users can use `dd` or image customization
- Today's Pi: **Raspberry Pi OS Lite 64bit (Bookworm)** — no GUI, CLI only

Ways to reach a Pi

- **SSH** — today's main method. Remote shell from terminal
- **VNC** — X11 desktop forwarding (not used today since Lite has no GUI)
- Reaching a Pi in a remote location:
  - ngrok / Cloudflare Tunnel — tunnel from NAT interior to outside
  - **Tailscale** — WireGuard-based mesh VPN. Modern standard
- 64-bit images support **VS Code Remote SSH**
  - VS Code Server auto-installs on the Pi side
  - Edit files on the Pi directly from your VS Code

Today we are in the same room, so we connect simply over the local network via SSH.

What is tile serving?

- Maps cannot be handled as "one image of the entire world"
  Tiles split by zoom level z and coordinates x,y are stitched together for display
- Client (MapLibre, etc.) requests only the tiles in the visible area
  `GET /tiles/{z}/{x}/{y}` — this is the "tile serving" mechanism
- Tiles come in two forms:
  - **Raster tiles** — pre-rendered images (PNG/JPEG/WebP)
  - **Vector tiles** — feature coordinates and attributes (MVT). Rendering happens client-side
- Vector tiles allow style changes later and attribute inspection. Today these are the star

Choosing a tile server

	Language/implementation	Traits	On 512MB Pi
GeoServer	Java	Feature-rich veteran. Serves WMS/WFS anything	Heavy
TileServer GL	Node.js	Style-inclusive delivery, rasterization	Somewhat heavy
pg_tileserv / Tegola	Go etc.	Assumes PostGIS	Needs DB
Martin	**Rust**	Lightweight, fast, single binary. PMTiles support	**Fit**

Why Martin:

- Single binary, place and run. No runtime or DB needed
- Small memory footprint (512MB is comfortable)
- **Can serve PMTiles directly**

What is PMTiles?

- An archive format holding millions of tiles in **one file** (by Protomaps)
  A single-file archive holding millions of tiles
- Internal indexing allows HTTP **Range requests** to read only needed tiles
  → works as a static file (S3, local disk, wherever)
- Properties that matter for today's workflow:
  - **Distribution is one file** — data updates done with one file copy
  - **No append, no DB** — kind to SD cards (read-centric)
  - Strong offline support — maps work without network if the file exists
- Similar format MBTiles uses SQLite inside. PMTiles is simple byte sequences readable with Range requests alone — the decisive difference

Today's datasets

File	Content	Size
hiroshima-base.pmtiles	Base map (Hiroshima city + Miyajima, z0–14)	15MB
shelters.pmtiles	Designated emergency evacuation sites	~1MB
hazard-flood.pmtiles	Flood inundation forecast areas (A31)	5MB
hazard-sediment.pmtiles	Landslide warning zones (A33)	14MB
terrain.pmtiles	Terrain RGB (5m DEM)	38MB

- Base map **generated locally with planetiler** (from OSM data in ~6 min)
  Basemap generated locally with planetiler (~6 min from OSM data)
- Generation and conversion are fully reproducible with scripts 01–04 in the data repository
- Contents, sources, and licenses of each tile: **catalog.html**
  Pipeline documentation: **data-pipeline.md**

Data sources and attribution

- © OpenMapTiles © OpenStreetMap contributors
- 「指定緊急避難場所データ」(国土地理院) をもとに作成
  Derived from "Designated Emergency Evacuation Site Data", GSI Japan
- 「国土数値情報 (洪水浸水想定区域データ)」(国土交通省) をもとに作成
  Derived from "National Land Numerical Information (Flood Inundation Zones)", MLIT Japan
- 「国土数値情報 (土砂災害警戒区域データ)」(国土交通省) をもとに作成
  Derived from "National Land Numerical Information (Landslide Warning Zones)", MLIT Japan
- 「基盤地図情報 数値標高モデル」(国土地理院) をもとに作成
  Derived from "Fundamental Geospatial Data (DEM)", GSI Japan

Attribution stays visible even in offline delivery — the bundled viewer auto-displays based on served sources.

Power up your Pi

Each seat's Pi has a pre-imaged SD card ready to go.

1. Confirm your seat's Pi and power adapter (labeled with seat number)
2. Insert microUSB and **power on** — there is no power switch. Insertion = startup
3. LED guide:
    - Red (PWR): power. Solid is normal
    - Green (ACT): SD card access. Blinks during startup
4. In 1–2 minutes the Pi joins the network and waits for SSH

No monitor or keyboard attached. Everything from here is over the network.
Instructor will confirm all units are up during the break.

Get your SSH client ready

Start your SSH client now.

- macOS / Linux: terminal works (`ssh` included)
- Windows: PowerShell / Windows Terminal `ssh`, or PuTTY
- Quick check:

```
ssh -V
```

Version output means you are ready. Raise your hand during the break if you have trouble.

Break (14:50–15:00)

After the break, we connect to the Pis.

PMTiles, locally first

Before the server, let's feel how "PMTiles is one file that becomes a map."

1. Get the provided `shelters.pmtiles`
2. Open **pmtiles.io** in your browser
3. Drag and drop the file

→ Evacuation shelters appear on the map without a server.
The viewer just reads the index inside the file directly.

Now we will do the same thing over the network.

Seat labels and SSH

- Each seat label: **seat number NN = last digits of IP address**
  Seat number NN = last part of the IP address
- IP follows `10.x.x.1NN` (seat 03 → last digits 103)

```
ssh workshop@10.x.x.1NN
```

- First time prompts for known_hosts check → `yes`
- Password is on the seat label
- Once connected:

```
hostname        # should output pi-NN
ip a            # confirm your IP
```

If trouble, we will help in the rescue time later.

It runs as a service

```
systemctl status martin
```

- `active (running)` — Martin is **a systemd service**
- What "runs as a service" means:
  - Auto-starts at boot (no login needed)
  - Auto-restarts if it crashes (`Restart=on-failure`)
  - Logs centralized in journald
- Difference from manual `./martin` is what enables "unattended recovery" in Part 3
  This is what makes unattended recovery possible in Part 3

Talking to Martin over HTTP

```
curl http://localhost:3000/health
curl http://localhost:3000/catalog | jq
```

- `/health` — liveness check. Basic monitoring
- `/catalog` — JSON list of served tile sources
- `jq` is a JSON formatting and extraction tool. Read it as:
  - Key: `"tiles"` holds each source
  - `content_type` and `description` hint at contents

"Check with curl before the browser" is basic server operations.

Under the hood

```
ls -lh /opt/tiles/
cat /opt/tiles/config.yaml
```

- PMTiles files and Martin configuration live here
- Try a live log stream:

```
journalctl -u martin -f
```

- Open the map in your browser and **watch your access appear in the log**
- `-u martin` narrows to the service, `-f` follows (tail)

The map, served by your Pi

Open in browser:

```
http://10.x.x.1NN/
```

- The **same data** you saw on pmtiles.io now comes **from your Pi**
- Check DevTools Network tab:
  - Tile request origin is `10.x.x.1NN`
  - Individual tile fetches visible
- Viewer is single-page HTML on MapLibre GL JS. The Pi serves this too

Click to inspect

- Click a shelter on the map → attributes pop up
- Vector tiles hold **data**, not pixels, so per-feature attributes are available
  Vector tiles carry attributes, not pixels
- Attributes to notice:
  - name — facility name
  - Disaster type — which disasters this shelter handles
- Later we will overlay whether a shelter works for flood scenarios

The swap exercise

Experience how "one file" updates work.

```
sudo mv /opt/tiles/hazard.pmtiles.bak /opt/tiles/hazard.pmtiles
sudo systemctl restart martin
curl http://localhost:3000/catalog | jq
```

- Browser refresh → **flood inundation layer appears**
- Notice the **overlap** of shelters and flooded areas:
  - Which shelters sit in flood zones?
  - Does it match the "handles floods" attribute?
- Field data updates are the same: place file and restart
- Flood vs. sediment differences: see docs/hazard-comparison.md in the data repository

Catch-up time

We prioritize people who need help. Raise your hand or use Slack.

For those moving ahead:

- Pick one tile in DevTools Network tab:
  - What is the **Content-Type**?
  - How big is **one tile** (tens of KB range)?
  - How does size vary by zoom level?
- CLI check with `curl -sI`:

```
curl -sI http://localhost:3000/hazard/10/900/400 | head
```

Aside: DEM archaeology

- Japan's DEM (digital elevation model) releases sparked an **archaeological discovery boom**
- Rendering 5m-precision DEM as hillshade reveals subtle ground features:
  - Medieval castle walls and baileys
  - Pre-Edo tombs and earthen mounds
- Forests hide nothing — airborne lidar "sees through" tree canopy to the ground
- Cultural heritage researchers are well represented at this FOSS4G.
  Ask them at social events — fascinating stuff
- Today's terrain.pmtiles comes from the same 5m DEM

Break (15:50–16:00)

The main event is next.

Going offline

- Part 3 uses **no internet in this room**
- Consult the downloaded PDF or **Pi's own clone** of the materials
  (the Pi serves the full documentation)

Diagram of AP mode:

```
Before:  PC → Opal router → Pi        (Pi is client)
Now:     Smartphone → Pi (itself as AP)     (Pi is the network parent)
```

- **One-way switch**: once AP, the Pi stays AP even after reboot
- **Your SSH will drop, and that is normal**
  Your SSH session will drop. That is expected, not a failure.

Flip the switch

Everyone runs this together:

```
sudo /opt/scripts/switch-to-ap.sh
```

- SSH drops (expected, as noted)
- 1–2 minutes later, SSID **`foss4g-pi-NN`** appears in Wi-Fi lists
  (NN is your seat number)
- What the script does:
  - Disables auto-connect to Opal (client mode)
  - Enables AP profile with auto-connect **on**
  - From now on, reboot brings up AP unconditionally

Receive it on your phone

1. Read your seat's **QR code** with your phone → connects to your Pi's AP
2. Open in browser:

```
http://192.168.4.1/
```

3. Same map appears — but now there is **no internet anywhere in the path**
   The same map — with no internet anywhere in the path

- `192.168.4.1` is the Pi's own address in AP mode
- Unlimited devices can connect. Connect with your neighbors

Rescue time

- If your AP did not come up, pair with a neighbor
  If your AP did not come up, please pair with a neighbor
- For those with a working AP:
  - Try connecting to a **neighbor's AP** too (`foss4g-pi-MM`)
  - Verify same viewer works from different Pis
- Each Pi uses 5GHz separate channels to avoid congestion
  (reveal later)

The main event

We are about to **cut power to the upstream Opal router**.

- Upstream vanishes. Internet path is completely gone
- Yet —

The map keeps working.
The map keeps working.

- Because it lives between your phone and Pi alone
- This is the moment the "portable tile server" idea becomes real

The power-pull exercise

Yank the server power suddenly. Field reality.

1. **Pull the microUSB from your Pi** (no shutdown command)
2. Wait 10 seconds and reinsert
3. Wait 1–2 minutes → SSID `foss4g-pi-NN` **comes back on its own**
4. Reconnect phone → map returns

- Nobody touched anything = **unattended recovery**
- Looks rough but the setup is built to handle this
  (read-centric PMTiles + minimized-write OS configuration)

Why it comes back

Two systems enable recovery:

systemd

- Martin is registered as a service → auto-starts at boot
- Crashes auto-restart with `Restart=on-failure`

NetworkManager auto-connect profile

- AP profile has autoconnect enabled → AP stands when booting
- The "one-way switch" is actually this autoconnect flag swap

Power on → OS boots → AP rises → Martin starts → delivery resumes.
Nobody intervenes.

Confirm all SSIDs are back. Raise your hand if not.

Designing for the field

Considerations when bringing today's setup to the site:

- **Connection UX** — QR code guides SSID connection. Captive-portal-style redirect can take you from "connected" to "map open" automatically
- **Health monitoring** — poll `/health` regularly. The `check-all.sh` script I ran during breaks does exactly this (scan all units)
- **Power** — 3W means half a day on mobile battery is realistic. Solar too
- **SD card durability** — disabled swap and tmpfs logs reduce writes (today's setup)
- **Radio planning secrets** — today we used 5GHz W52 band spread across channels 36/40/44/48 by seat cluster. 15 units dense on 2.4GHz breaks down

Aside: supply chain was the bottleneck

The biggest constraint in workshop prep was not technology — it was **sourcing**.

- Data center demand triggered global shortages hitting end-user gear
- Even MacBook RAM is scarce; MacBook Air cannot get 64GB configs. Mac mini too
- Ripple effect on microcontrollers: **Raspberry Pi Zero 2 W was hard to source in Japan until November**
- Field lesson learned: **source equipment one semester before you need it**

Take-home extras

Procedures in extra.md; actual code in the data repository's `scripts/` and data-pipeline.md.
All reproducible at home:

- **Tile generation (PC)** — planetiler generates basemap from OSM (~6 min)
- **DEM → Terrain RGB** — build terrain tiles from Fundamental Geospatial Data DEM
- **Maputnik** — browser-based style editor to customize appearance
- **autohotspot** — auto-switching between client and AP (today was one-way)
- **Swap to sediment** — swap with hazard-sediment.pmtiles.bak for another round
  (same as flood exercise: mv, restart)

Q&A

- Raise your hand or post to Slack. We will pick up backlog questions here too
- Contact and repository links come up again in the closing

Wrap-up: four takeaways

1. **Portable as a box** — server and maps fit in one hand-sized box
   The whole stack fits in one hand
2. **Zero device prep** — deliver to any device that connects
   Client devices need zero preparation
3. **Updates are one file** — mv and restart finish the job
   Updating means swapping one file
4. **Unattended recovery** — pull power and it comes back alone
   It recovers with nobody watching

Closing

- **Pi collection** — we will collect, matched by seat. Thank you for your help
- **Survey** — QR displayed. Feedback for the organizers
- **Slack stays open** — post questions anytime later
  The Slack workspace stays open
- **Repository** — slides, scripts, and data generation procedures all here:

  github.com/alt9800/Raspi-Geo-Workshop

Thank you — and may your maps work where the network doesn't.